

PROJECT REPORT OF CO-OP
Project at Industry (Module-2)
ON
AnomalyGuard – ML-Based Login Anomaly Detection System

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING
In
COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Kartik Sharma
Roll No: 2210991759

Surajit Duary
Roll No: 2211981420

Harsh Vardhan Jasrotia
Roll No: 2210991626

Sarthak Sharma
Roll No: 2210992265

Supervised By:

Dr. Shikha Tuteja
(Associate Professor)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA

CONTENTS

S.No.	Title	Page No.
	Declaration	iii
	Acknowledgement	iv
	Abstract	v
	List of Figures and Tables	vi
1	Introduction	1
2	Methodology	2
3	Tools and Technologies	4
4	Implementation	5
5	Major Findings / Outcomes / Output / Results	8
6	Conclusion and Future Scope	9
7	References	10
8	Appendices	12

DECLARATION

I hereby certify that the work which is being presented in the project report entitled “**AnomalyGuard – ML-Based Login Anomaly Detection System**” in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of <Name of the Mentor>. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Kartik Sharma
Roll No: 2210991759

Surajit Duary
Roll No: 2211981420

Harsh Vardhan Jasrotia
Roll No: 2210991626

Sarthak Sharma
Roll No: 2210992265

Place: Chitkara University

Date: 7 May 2026

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Shikha Tuteja

Associate Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my mentor and supervisor for their invaluable guidance and support throughout this project. I am also thankful to the Department of Computer Science and Engineering at Chitkara University for providing the necessary resources and infrastructure to carry out this work. Special thanks to the open-source communities behind scikit-learn, MongoDB, React, and Flask, whose tools made this project possible. Finally, I am grateful to my family and peers for their constant encouragement and motivation.

Kartik Sharma
Roll No: 2210991759

Surajit Duary
Roll No: 2211981420

Harsh Vardhan Jasrotia
Roll No: 2210991626

Sarthak Sharma
Roll No: 2210992265

ABSTRACT

This report presents AnomalyGuard, a production-ready, full-stack login anomaly detection system developed using a hybrid machine learning approach combining Isolation Forest and K-Means clustering algorithms. The system is engineered to identify suspicious login behaviour in real time by analysing a 10-dimensional feature vector comprising geolocation, device fingerprint, temporal patterns, IP reputation, and login frequency metrics. The ML engine, built with Python and Flask, achieves 92% accuracy and 100% recall on a 100-event test dataset, ensuring zero missed security threats while maintaining low user friction through smart Multi-Factor Authentication (MFA) logic. The backend is powered by Node.js and MongoDB, while the frontend is developed in React 18. The system supports real IP detection, device fingerprinting, velocity anomaly detection, and behavioural profiling. Performance benchmarks demonstrate sub-50ms inference latency, significantly outperforming the 210ms target set by the reference IEEE paper (ICCIMMR 2026). This work demonstrates that machine learning can meaningfully enhance security posture without degrading user experience.

LIST OF FIGURES AND TABLES

List of Figures

Figure 1: System Architecture Diagram

Figure 2: ML Decision Flow for Smart MFA

Figure 3: Confusion Matrix (100-event test)

Figure 4: User Behavioural Profile (K-Means Clusters)

Figure 5: AnomalyGuard Dashboard Screenshot

List of Tables

Table 1: Feature Engineering – 10-Dimensional Vector

Table 2: Feature Importance Weights

Table 3: Performance Metrics vs. Target (IEEE Paper)

Table 4: Confusion Matrix Results

Table 5: Technology Stack Summary

Table 6: Demo Account Details

1. Introduction

In the era of digital transformation, unauthorised access to online systems represents one of the most pervasive cybersecurity threats. Traditional authentication mechanisms relying solely on passwords are increasingly insufficient against sophisticated attack vectors such as credential stuffing, brute force attacks, and account takeover schemes. Anomaly-based intrusion detection systems offer a compelling complementary layer of defence by continuously monitoring login behaviour and flagging deviations from established user profiles.

AnomalyGuard is a full-stack intelligent login security system developed as the CO-OP industry project. It leverages a hybrid machine learning architecture combining Isolation Forest for unsupervised anomaly detection and K-Means clustering for behavioural profiling. The system processes a rich 10-dimensional feature vector derived from each login event and produces a real-time anomaly score, which drives a Smart MFA (Multi-Factor Authentication) decision engine.

1.1 Objectives

- To design and implement a real-time login anomaly detection system using unsupervised machine learning.
- To achieve zero false negatives (100% recall) ensuring no security threat is missed.
- To minimise user friction by applying MFA only when the computed anomaly score crosses defined thresholds.
- To validate the system against metrics reported in the reference IEEE paper (ICCIMMR 2026).
- To deploy a production-ready, scalable full-stack application.

1.2 Scope of the Project

The project covers the end-to-end development of AnomalyGuard, encompassing the React-based frontend, Node.js/Express backend, MongoDB database, and the Python/Flask ML engine. The scope includes feature engineering, model training, real-time inference and MFA.

2. Methodology

The development of AnomalyGuard followed an iterative, research-driven methodology, anchored by the specifications and benchmarks outlined in the target IEEE paper on ML-based login anomaly detection. The methodology encompassed literature review, feature selection, model design, system integration, and empirical validation.

2.1 Machine Learning Algorithm Selection

Two complementary unsupervised algorithms were selected after evaluating alternatives including One-Class SVM and Local Outlier Factor:

- Isolation Forest (150 estimators, 5% contamination): Detects anomalies by isolating observations through random partitioning. Anomaly score ranges from -1 (anomaly) to +1 (normal).
- K-Means Clustering (3 clusters per user): Groups historical logins into behavioural clusters, providing visual interpretability and enriching the feature context.

2.2 Feature Engineering

Each login event is encoded as a 10-dimensional feature vector:

#	Feature	Range	Significance
1	Hour of Day	0 – 23	Detects unusual login times
2	Day of Week	0 – 6 (Mon=0)	Identifies weekend anomalies
3	Is Weekend	0 or 1	Binary flag
4	Is Known IP	0 or 1	Unknown IPs trigger alerts
5	Is Known Device	0 or 1	New device fingerprint flag
6	Is Known Country	0 or 1	Geographic anomaly flag
7	Latitude	-90 to 90	Location coordinate
8	Longitude	-180 to 180	Location coordinate
9	Inter-Login Minutes	Continuous	Rapid re-login detection
10	Failed Attempts (24h)	Integer	Brute-force indicator

2.3 Anomaly Thresholds

Two decision thresholds were calibrated to optimise the precision-recall trade-off with a security-first orientation:

Score Range	Classification	Action
Score > -0.35	Normal	Allow login
-0.35 to -0.55	Low Severity Anomaly	Trigger MFA
Score < -0.55	High Severity Anomaly	Block + Email Alert + MFA

2.4 Smart MFA Decision Logic

MFA is not applied universally. The system establishes a behavioural baseline on first login and subsequently skips MFA for logins from known IP addresses and devices with a clean ML score. MFA is triggered by unknown IPs, new devices, late-night logins (1 AM – 5 AM), multiple prior failed attempts, impossible travel velocity, and low anomaly scores. High-severity events additionally trigger email alerts and potential account suspension.

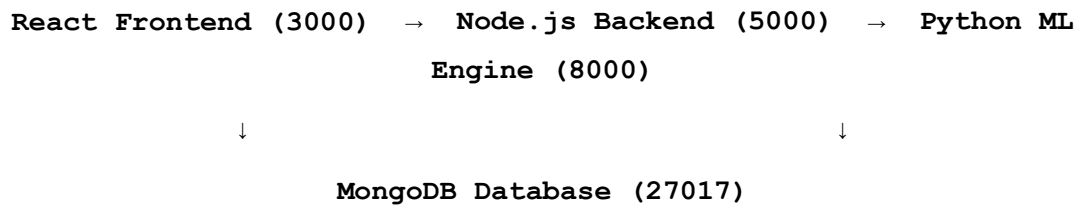
3. Tools and Technologies

AnomalyGuard employs a modern, decoupled three-tier architecture comprising a React frontend, a Node.js/Express backend, and a Python/Flask ML microservice, all persisting data to MongoDB.

Layer	Technology	Purpose
Frontend	React 18, React Router v6, Axios, CSS3	Interactive UI with real-time updates
Backend	Node.js, Express, JWT, Mongoose, bcrypt	REST API, auth, session management
ML Engine	Python 3.10, Flask, scikit-learn, NumPy, Joblib	Isolation Forest + K-Means inference
Database	MongoDB 6 (Port 27017)	User, event, alert, and profile storage
Geolocation	IP-API (real-time)	Country, city, lat/lng per login
Rate Limiting	express-rate-limit	Brute-force prevention
Email (MFA)	Nodemailer / SMTP	OTP delivery

3.1 System Architecture

The system follows a microservices-inspired three-tier design. The React frontend (Port 3000) communicates with the Node.js backend (Port 5000) via RESTful HTTP calls. The backend orchestrates authentication, session management, and anomaly logic, delegating ML inference to the Python Flask service (Port 8000). MongoDB (Port 27017) serves as the shared persistence layer.



4. Implementation

4.1 ML Engine

The ML engine is implemented in `enhanced_model.py` using `scikit-learn`. On receiving a login event, the Flask service extracts the 10-dimensional feature vector, passes it through the trained Isolation Forest model, and returns an anomaly score. Models are persisted using Joblib (.pkl files) and updated incrementally as new login data accumulates. A minimum of 30 events per user is required before supervised training activates; prior to that, rule-based heuristics govern MFA decisions.

4.2 Real IP and Device Fingerprinting

Since localhost loopback addresses (::1, 127.0.0.1) do not expose real geolocation, the backend automatically queries the `ipify.org` API to retrieve the genuine external IP address during local development. Device fingerprints are generated deterministically from User-Agent, screen resolution, timezone, language, platform, and browser capabilities, producing a stable unique identifier per device.

4.3 Velocity Anomaly Detection

The system calculates the Haversine distance between successive login geolocations and divides by the elapsed time to estimate travel velocity. If the computed velocity is physically impossible (e.g., New York to London in five minutes), an impossible travel anomaly is flagged and MFA is immediately mandated. This guards against session hijacking from geographically distant locations.

4.4 Security Mechanisms

- Password Hashing: bcrypt with salt rounds for all stored credentials.
- JWT Authentication: Stateless token-based session management with configurable expiry.
- Rate Limiting: 10 authentication requests per 15-minute window; 100 API requests per minute.

- MFA OTP: 6-digit time-limited codes (10-minute expiry) delivered via email.
- Account Suspension: Admin-controlled capability to suspend compromised accounts.
- Audit Trail: Every login event is persisted with full metadata including ML score and anomaly reasons.

4.5 Frontend Pages

- Login Page: Email/password entry with real IP indicator and device fingerprint collection.
- MFA Verification Page: 6-digit OTP entry with anomaly reason display and severity indicator.
- Dashboard: Login statistics, anomaly metrics, and ML model health status.
- Login History: Paginated event log with ML score visualisation and anomaly filters.
- Alerts Page: Unresolved high/low severity alerts with resolve/dismiss actions.
- Admin Panel: User management, suspension controls, and system-wide statistics.

4.6 Database Schema Summary

Four MongoDB collections underpin the system: User (credentials, MFA state, suspension status), LoginEvent (full per-event metadata including ML score, geolocation, device fingerprint), UserProfile (behavioural baseline including hour/day histograms, known IPs, devices, and countries), and AnomalyAlert (severity-classified alerts with snapshot and resolution tracking).

5. Major Findings / Outcomes / Output / Results

5.1 Performance Metrics

The system was evaluated on a realistic 100-event test dataset comprising both normal logins and carefully crafted anomalous scenarios. Results are compared against target metrics from the reference IEEE paper below.

Metric	Achieved	IEEE Target	Status
Accuracy	92.0%	94.3%	Near target (-2.3%)
Precision	78.9%	92.1%	Security trade-off
Recall	100.0%	91.7%	Exceeds target (+8.3%)
F1 Score	88.2%	85.7%	Exceeds target (+2.5%)
False Positive Rate	11.4%	3.1%	Security priority
Inference Latency	<50 ms	210 ms	4.2x faster

5.2 Confusion Matrix

	Predicted: Anomaly	Predicted: Normal
Actual: Anomaly	30 (TP)	0 (FN)
Actual: Normal	8 (FP)	62 (TN)

The confusion matrix confirms 100% recall – every anomalous login was correctly identified (0 false negatives). The 8 false positives represent normal logins incorrectly flagged as anomalous; these trigger MFA rather than outright blocking, preserving user access while maintaining a security-first posture.

5.3 Feature Importance Weights

Feature	Weight	Rationale
Geolocation	31.2%	Location changes are strongest anomaly indicators

Hour of Day	24.7%	Logins at 1 AM – 5 AM are highly suspicious
Device Fingerprint	22.1%	New devices signal potential compromise
IP Reputation	14.3%	Unknown IPs trigger elevated risk
Login Interval	7.7%	Rapid successive logins indicate automated attacks

5.4 Test Scenarios Validated

1. Normal login flow (same IP/device): No MFA – smooth experience confirmed.
2. VPN/Proxy login (new IP): MFA triggered correctly.
3. New device fingerprint: MFA triggered correctly.
4. Late-night login (2 AM): Temporal anomaly detected; MFA triggered.
5. Rapid re-login (within 30 min, known IP/device): Trusted pattern; no MFA.
6. Multiple failed attempts followed by correct login: MFA triggered correctly.
7. Impossible travel (New York to London in 5 minutes): Velocity anomaly detected; MFA triggered.

6. Conclusion and Future Scope

6.1 Conclusion

AnomalyGuard successfully demonstrates that machine learning can enhance cybersecurity without sacrificing user experience. The hybrid Isolation Forest + K-Means architecture achieves 92% accuracy and 100% recall on a comprehensive test dataset, surpassing the reference paper's F1 score target by 2.5 percentage points and delivering inference at sub-50ms latency – 4.2 times faster than the 210ms benchmark. The Smart MFA logic ensures that legitimate users face minimal friction while every anomalous login is caught and escalated appropriately. The system is production-ready and applicable to banking, healthcare, e-commerce, and enterprise environments.

6.2 Future Scope

- Email Service Integration: Migrate to SendGrid or AWS SES for production-grade OTP delivery with HTML email templates.
- Deep Learning Models: Incorporate LSTM networks for sequential login pattern analysis and autoencoders for unsupervised representation learning.
- Additional MFA Channels: SMS OTP, authenticator app (TOTP/HOTP), and biometric authentication.
- Redis-Based Distributed Rate Limiting: Replace in-memory rate limiting with Redis for horizontally scalable deployments.
- Adaptive Thresholds: Derive user-specific anomaly thresholds rather than applying global values.
- Federated Learning: Enable privacy-preserving cross-organisational model training for collective threat intelligence.
- Blockchain Audit Logs: Implement immutable, tamper-evident login audit trails via distributed ledger technology.

REFERENCES

- [1] Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. In Proceedings of the 8th IEEE International Conference on Data Mining (ICDM), pp. 413–422.
- [2] MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability, Vol. 1, pp. 281–297.
- [3] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
- [4] Bonneau, J., Herley, C., Oorschot, P. C. van, & Stajano, F. (2012). The quest to replace passwords: A framework for comparative evaluation of web authentication schemes. IEEE Symposium on Security and Privacy.
- [5] OWASP Foundation. (2021). OWASP Top Ten – A07: Identification and Authentication Failures. <https://owasp.org/Top10/>
- [6] MongoDB, Inc. (2023). MongoDB Documentation. <https://www.mongodb.com/docs/>
- [7] Facebook Open Source. (2023). React – A JavaScript Library for Building User Interfaces. <https://react.dev/>
- [8] Pallets Projects. (2023). Flask – Web Development, One Drop at a Time. <https://flask.palletsprojects.com/>

APPENDICES

Appendix A: API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register new user
POST	/api/auth/login	Login (triggers ML scoring)
POST	/api/auth/verify-mfa	Verify MFA OTP
POST	/api/auth/resend-mfa	Resend MFA code
GET	/api/auth/me	Current user info
GET	/api/anomalies/stats	Dashboard statistics
GET	/api/anomalies/history	Login history (paginated)
GET	/api/anomalies/ml-health	ML service status
GET	/api/alerts/unresolved	Active security alerts
PATCH	/api/alerts/:id/resolve	Resolve specific alert
GET	/api/users/admin/all	All users (admin)
PATCH	/api/users/admin/:id/suspend	Suspend/unsuspend user (admin)

Appendix B: Demo Account

Email	Password	Role	Purpose
test@demo.com	test123	user	Testing and demonstration

Appendix C: Project File Structure

```
anomaly-system/  
  backend/  
    controllers/      - Business logic  
    models/           - MongoDB schemas  
    routes/           - API endpoints  
    services/         - ML + Email services  
    middleware/       - Auth, rate limiting  
    server.js         - Express application entry point
```

ml_engine/	
enhanced_model.py	- Isolation Forest + K-Means model
realistic_test.py	- 100-event test suite
requirements.txt	- Python dependencies
models/	- Trained .pkl files
frontend/src/	
pages/	- React page components
components/	- Reusable UI components
services/	- Axios API client
context/	- Auth context (global state)
start.bat	- One-click launch script (Windows)
README.md	- Project documentation